SVKM's

School of Technology Management & Engineering

NMIMS, Indore

Azile & Product Development

Project Report

Name: Krishka Kate (A013), Abhimanyu Nema (A004)

SAP Id: 70562300089, 70562300038

Project Topics: Your Store-Quick Commerce App for Local Clothing Stores

Class: B. Tech AI & DS

Semester: 6

GitHub Link: https://github.com/abhimanyu284/Your-Store.git

Submitted to: Dr. Gaurav Paliwal

1. Problem Statement

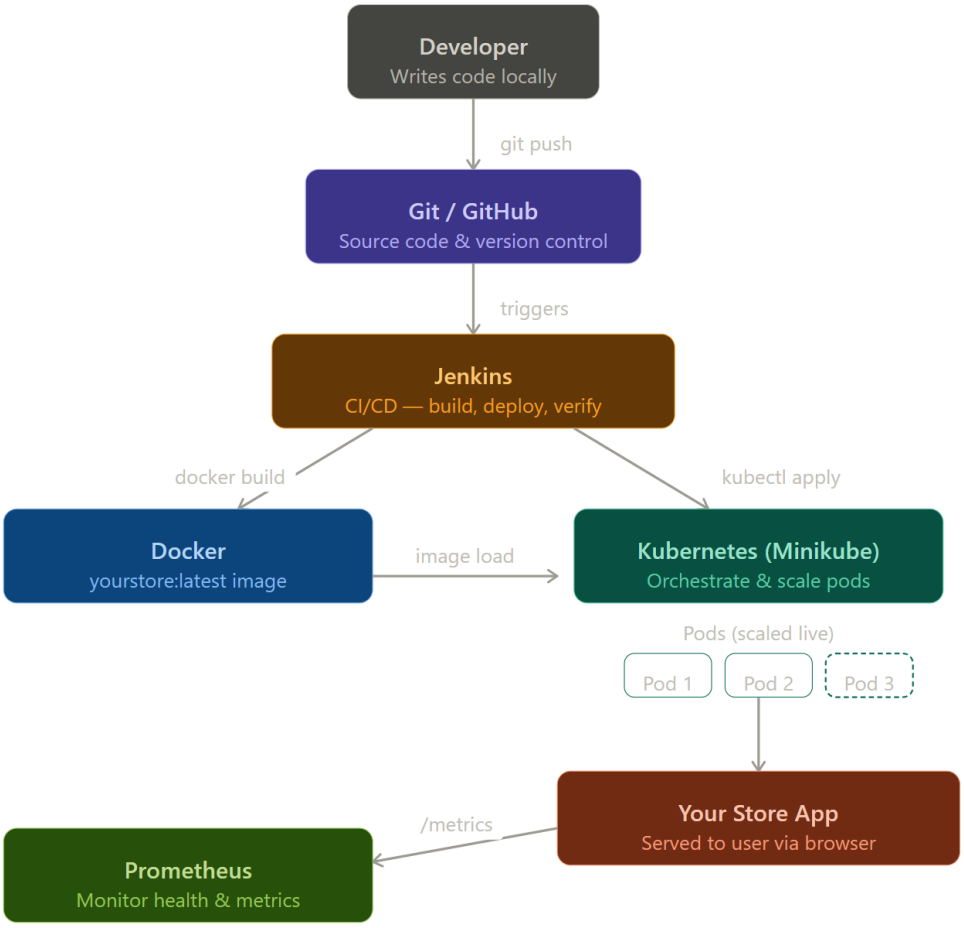
Local clothing stores in India lack a digital presence and fast delivery infrastructure. Customers prefer large e-commerce platforms, which pushes local stores out of business. **Your Store** solves this by giving local stores a unified digital storefront with quick commerce delivery — combining the speed of Zepto/Blinkit with the inventory of nearby fashion stores. The DevOps pipeline ensures this platform can be deployed, updated, and monitored reliably at any scale.

2. Objectives

- Build a functional quick commerce web application for local clothing stores
- Containerize the application using Docker for environment consistency
- Orchestrate containers using Kubernetes (Minikube) with live scaling capability
- Automate the build-deploy cycle using Jenkins CI/CD pipeline
- Monitor application health and performance using Prometheus
- Demonstrate a complete, real-world DevOps workflow from code push to production

3. System Architecture (Text Explanation)

The architecture is divided into four layers:



Your Store — App Description & Project Overview

What is Your Store?

Your Store is a **quick commerce application for local clothing stores**, built on the same model as Zepto and Blinkit — but instead of groceries, it delivers *fashion*. The idea is simple: users can browse clothing from nearby local stores and get items delivered fast.

The app serves multiple local stores like **TrendZone**, **FabFashion**, and **UrbanWear**, giving them a unified digital storefront.

App Features

- Browse products by category — Men, Women, Kids, Unisex, Ethnic, Sale
- Add items to cart and place orders
- Real-time order tracking with delivery ETA
- Products served from multiple local stores on one platform

Tech Stack

Frontend — A single-page web app built with HTML, CSS, and JavaScript. It includes product listings, category filters, a cart system, and an order tracking screen.

Backend — Node.js with Express.js, exposing REST APIs for products, categories, cart, and orders. It also has a /health endpoint and a /metrics endpoint for monitoring.

What Are We Demonstrating?

The core goal of this project is to show a **complete, real-world DevOps pipeline** applied to a working application. Here's what each tool contributes:

Git & GitHub — All source code is version-controlled and pushed to a GitHub repository. This simulates how real development teams manage and collaborate on code.

Docker — The app is containerized using Docker, meaning it's packaged with all its dependencies into an image called yourstore:latest. This ensures the app runs identically on any machine, no environment issues.

Kubernetes (Minikube) — Kubernetes orchestrates the Docker containers. It handles deploying the app, keeping it running, and scaling it. For the demo, you can visually show pods scaling from 1 to 3 replicas live with a single command.

Jenkins — Jenkins automates the entire pipeline. Whenever code is pushed to GitHub, Jenkins automatically pulls it, builds the Docker image, loads it into Minikube, and deploys it — no manual steps needed. This is the CI/CD pipeline.

Prometheus — Prometheus scrapes the /metrics endpoint on the app and monitors its health and performance in real time. This simulates production-level observability.

Layer 1 — Developer & Source Control: The developer writes code locally and pushes it to GitHub. Git tracks every change. GitHub acts as the single source of truth for the codebase.

Layer 2 — CI/CD Automation (Jenkins): Jenkins is connected to the GitHub repository. When new code is pushed, Jenkins automatically triggers a pipeline that clones the repo, builds a Docker image, loads it into Minikube, and deploys it to Kubernetes — all without manual intervention.

Layer 3 — Container Orchestration (Kubernetes/Minikube): Kubernetes receives the deployment instructions and runs the Docker containers as Pods. A NodePort Service exposes the app to the browser. The deployment can be scaled from 1 to 3 replicas with a single command, demonstrating horizontal scaling.

Layer 4 — Monitoring (Prometheus): Prometheus runs as a separate Docker container and continuously scrapes the /metrics endpoint of the running app. This provides real-time visibility into request counts, response times, and app health.

Git/GitHub

What is Git?

Git is a version control system — it tracks every change made to your code over time. Think of it like a detailed history book for your project. Every time you save a version (called a "commit"), Git records what changed, when, and who changed it.

What is GitHub?

GitHub is a cloud-based platform that hosts your Git repository online. It's where your code lives remotely so it's safe, shareable, and accessible from anywhere.

Why are we using it?

In real companies, multiple developers work on the same codebase simultaneously. Git makes sure their changes don't clash. GitHub acts as the central source of truth for the code.

What we did in this project:

- Initialized a local Git repo with git init
- Committed all the app's source code
- Pushed it to GitHub at github.com/abhimanyu284/Your-Store
- This GitHub repo is later used by Jenkins to automatically pull the latest code whenever something changes

Real World Analogy

Git is like Google Docs version history — you can see every edit ever made and roll back to any previous version instantly.

Commands:

1. Git init

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/quickcommerce
-backend (main)
$ git init
Reinitialized existing Git repository in C:/Users/Abhimanyu Nema/OneDrive/Desktop/Projects/quickcommerce-backend/.git/
```

2. Git add .

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/quickcommerce-backend (main)
$ git add .
```

3. Git commit -m "Initial commit"

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/quickcommerce-backend (main)
$ git commit -m "Initial commit"
[main c4a2232] Initial commit
 3 files changed, 680 insertions(+), 1080 deletions(-)
```

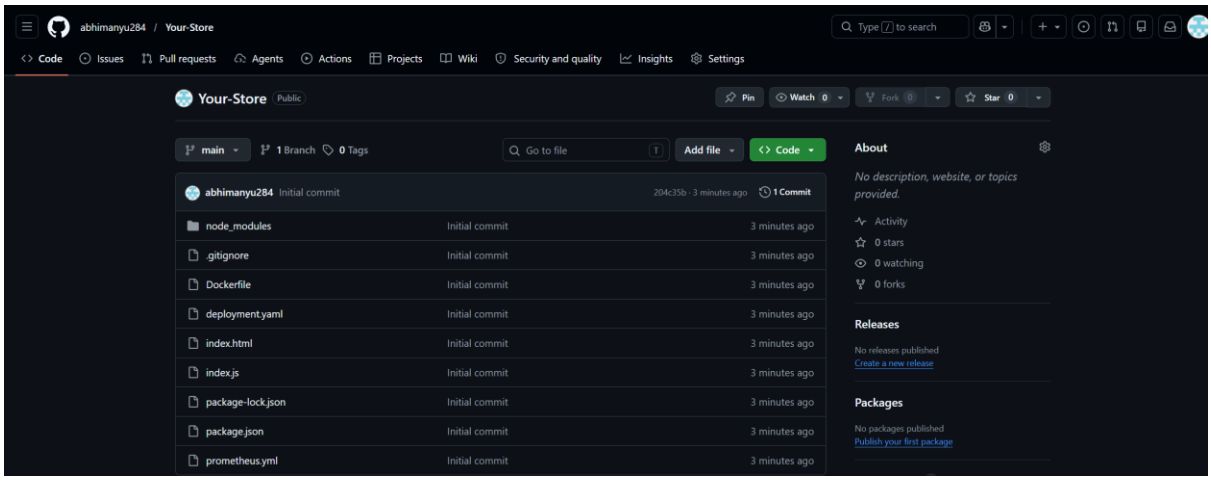
4. git remote add origin https://github.com/abhimanyu284/Your-Store

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/Your Store (main)
$ git remote add origin https://github.com/abhimanyu284/Your-Store.git
```

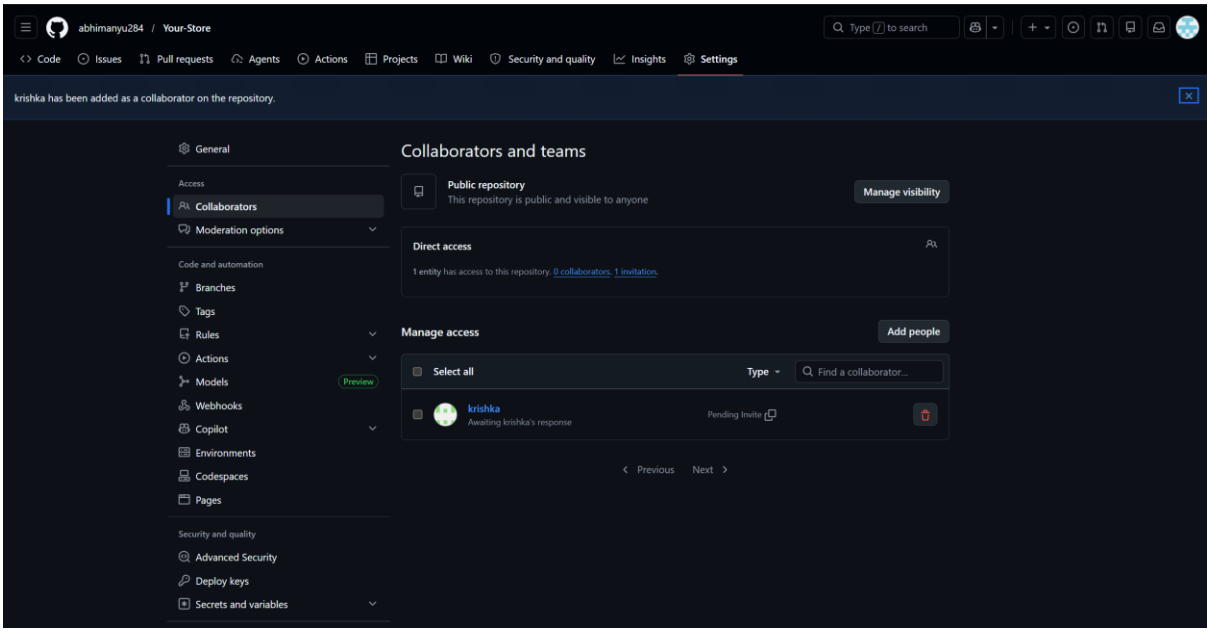
5. git push -u origin main –force

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/Your Store (main)
$ git push -u origin main --force
Enumerating objects: 1137, done.
Counting objects: 100% (1137/1137), done.
Delta compression using up to 20 threads
Compressing objects: 100% (1077/1077), done.
Writing objects: 100% (1137/1137), 1.92 MiB | 1.22 MiB/s, done.
Total 1137 (delta 277), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (277/277), done.
To https://github.com/abhimanyu284/Your-Store.git
 + b0f96b6...204c35b main -> main (forced update)
branch 'main' set up to track 'origin/main'.
```

Repository:



Adding Collaborators:



Docker

What is Docker?

Docker is a containerization platform. It packages your application along with everything it needs to run — the code, runtime, libraries, dependencies — into a single unit called a container.

What is a Container?

A container is like a lightweight, portable box that has your app inside it. No matter where you run it — your laptop, a server, the cloud — it behaves exactly the same way.

What is a Docker Image?

An image is the blueprint for a container. You build the image once, and you can run as many containers from it as you want. In our project, the image is called yourstore:latest.

Why are we using it?

Without Docker, the classic problem is *"it works on my machine but not on yours."* Docker eliminates this entirely because the environment is always identical.

What we did in this project

- Wrote a Dockerfile that defines how to build the app
- Built the image using docker build -t yourstore:latest .
- Tested the app locally by running it as a container on port 3000
- This same image is then handed over to Kubernetes for deployment

Real World Analogy

Docker is like a shipping container. No matter what ship, truck, or port it goes through — the contents inside are always intact and identical.

Commands:

1. docker build -t yourstore:latest .

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/Your Store (main)
$ docker build -t yourstore:latest .
[+] Building 21.3s (10/10) FINISHED                                                                                    docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                                   0.1s
 => => transferring dockerfile: 159B                                                                                                   0.0s
 => [internal] load metadata for docker.io/library/node:18-alpine                                                                      2.4s
 => [internal] load .dockerignore                                                                                                      0.1s
 => => transferring context: 2B                                                                                                        0.0s
 => [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e                0.1s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e                0.1s
 => [internal] load build context                                                                                                     15.2s
 => => transferring context: 8.53MB                                                                                                   15.1s
 => CACHED [2/5] WORKDIR /app                                                                                                          0.0s
 => CACHED [3/5] COPY package*.json ./                                                                                                 0.0s
 => CACHED [4/5] RUN npm install                                                                                                       0.0s
 => [5/5] COPY .                                                                                                                       0.9s
 => exporting to image                                                                                                                 1.9s
 => => exporting layers                                                                                                                0.9s
 => => exporting manifest sha256:420cc3a5cd5adadc2ce69434e46dcb0c18a6843d00b2478cfb780456014bc8af                                      0.0s
 => => exporting config sha256:a0ff3fb344bdf50e40f9f049a69a97b0ed3ecdf513f68b74bd8b3bb9444bd627                                       0.0s
 => => exporting attestation manifest sha256:2851e044a94291e20abacba46157925f6ac933c3f0fe08e3a9166512140709ad                          0.1s
 => => exporting manifest list sha256:eb4b9c57e8e925a238cbc30099f8365e791ebd60375a22f8d1e5fa09955477aa                                 0.0s
 => => naming to docker.io/library/yourstore:latest                                                                                    0.0s
 => => unpacking to docker.io/library/yourstore:latest                                                                                 0.8s
```

2. docker run -d -p 3000:3000 --name yourstore yourstore:latest

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/Your Store (main)
$ docker run -d -p 3000:3000 --name yourstore yourstore:latest
f1d781df7c2d0ca7160fd28dfe4bbf2a5b60dbecbccf8243349ee6ca264eed8f
```

3. docker stop yourstore

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/Your Store (main)
$ docker stop yourstore
yourstore
```

4. docker rm yourstore

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/Your Store (main)
$ docker rm yourstore
yourstore
```

Kubernetes

What is Kubernetes?

Kubernetes (also written as **K8s**) is a **container orchestration system**. It manages, deploys, scales, and monitors your Docker containers automatically.

What is Minikube?

Minikube is a **local version of Kubernetes** that runs on your own laptop. It lets you practice and demo Kubernetes without needing a cloud server. It's perfect for development and demonstrations.

Key Concepts:

Pod — The smallest unit in Kubernetes. A pod runs one or more containers. In our project, each pod runs one instance of the Your Store app.

Deployment — A set of instructions that tells Kubernetes how many pods to run, which image to use, and how to handle updates.

Service — Exposes your pods to the outside world so users can actually access the app through a browser.

Scaling — Kubernetes can automatically increase the number of pods when traffic is high and reduce them when it's low.

Why are we using it?

Docker alone can only run one container. Kubernetes manages dozens or hundreds of containers, keeps them healthy, restarts them if they crash, and balances traffic across them.

What we did in this project

- Started Minikube with minikube start
- Loaded our Docker image into Minikube
- Applied deployment.yaml to tell Kubernetes how to run the app
- Demonstrated **live scaling** — going from 1 pod to 3 pods with a single command
- Watched pods come up in real time with kubectl get pods -w

Real World Analogy

If Docker is the shipping container, Kubernetes is the **entire port management system** — deciding which ship carries which container, how many to load, and rerouting if something breaks.

Commands:

1. Minikube start

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/Your Store (main)
$ minikube start
* minikube v1.38.1 on Microsoft Windows 11 Home Single Language 25H2
* Using the docker driver based on existing profile
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Restarting existing docker container for "minikube" ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: default-storageclass, storage-provisioner
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

2. minikube image load yourstore:latest

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/Your Store (main)
$ minikube image load yourstore:latest
```

3. kubectl apply -f deployment.yaml

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/Your Store (m
ain)
$ kubectl apply -f deployment.yaml
deployment.apps/quickcommerce-api unchanged
service/quickcommerce-service unchanged
```

4. kubectl get pods

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/Your Store (m
ain)
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS        AGE
numberguess-865fbdc94-d44xh         1/1     Running   20 (2m27s ago)  22d
numberguess-865fbdc94-gdvpk         1/1     Running   20 (2m27s ago)  22d
numberguess-865fbdc94-ml6qh         1/1     Running   20 (2m27s ago)  22d
quickcommerce-api-bccddb575-8bsd5   1/1     Running   2 (2m27s ago)   5d14h
quickcommerce-api-bccddb575-c2b9t   1/1     Running   2 (2m27s ago)   5d14h
quizapp-97fd4b7f7-jn97w             1/1     Running   19 (2m27s ago)  14d
```

5. minikube service quickcommerce-service

```
Abhimanyu Nema@LAPTOP-7L5B3FEJ MINGW64 ~/OneDrive/Desktop/Projects/Your Store (main)
$ minikube service quickcommerce-service
```

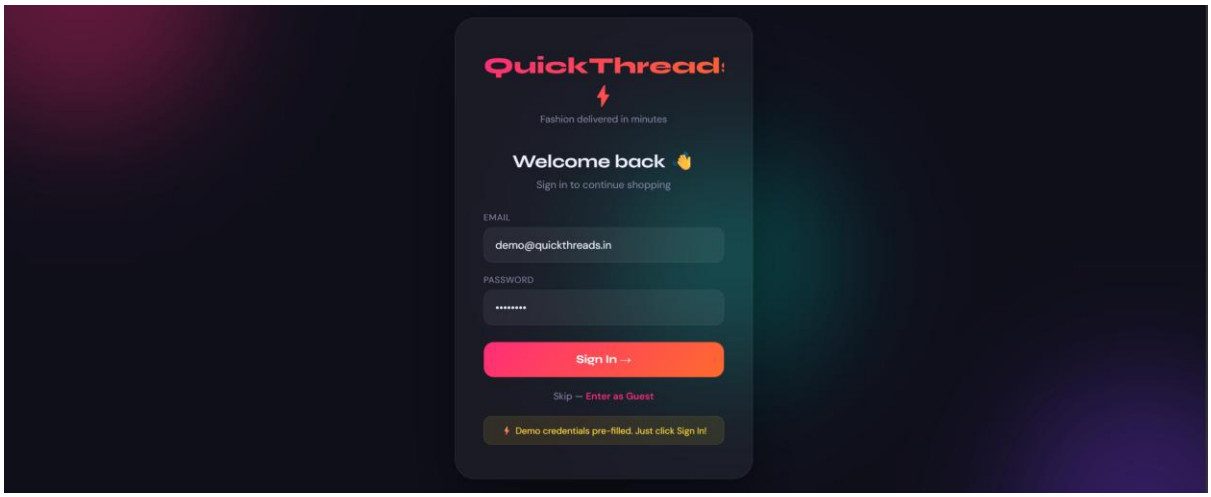
NAMESPACE	NAME	TARGET PORT	URL
default	quickcommerce-service	3000	http://192.168.49.2:30001

```
* Starting tunnel for service quickcommerce-service.
```

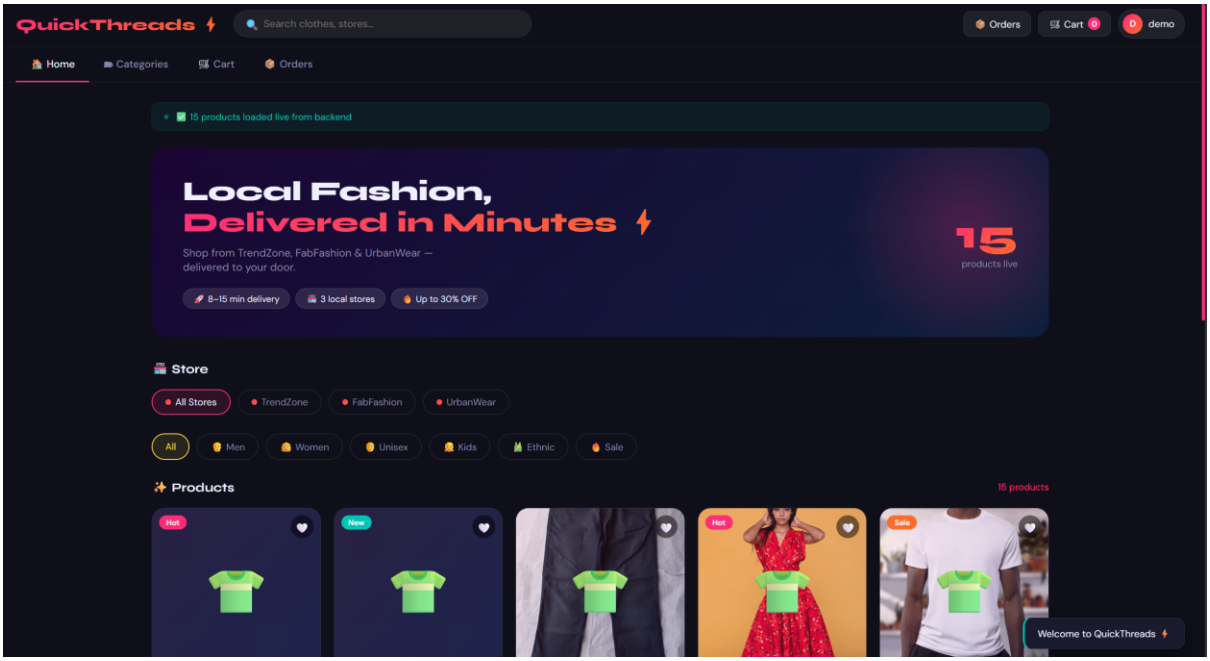
NAMESPACE	NAME	TARGET PORT	URL
default	quickcommerce-service		http://127.0.0.1:49246

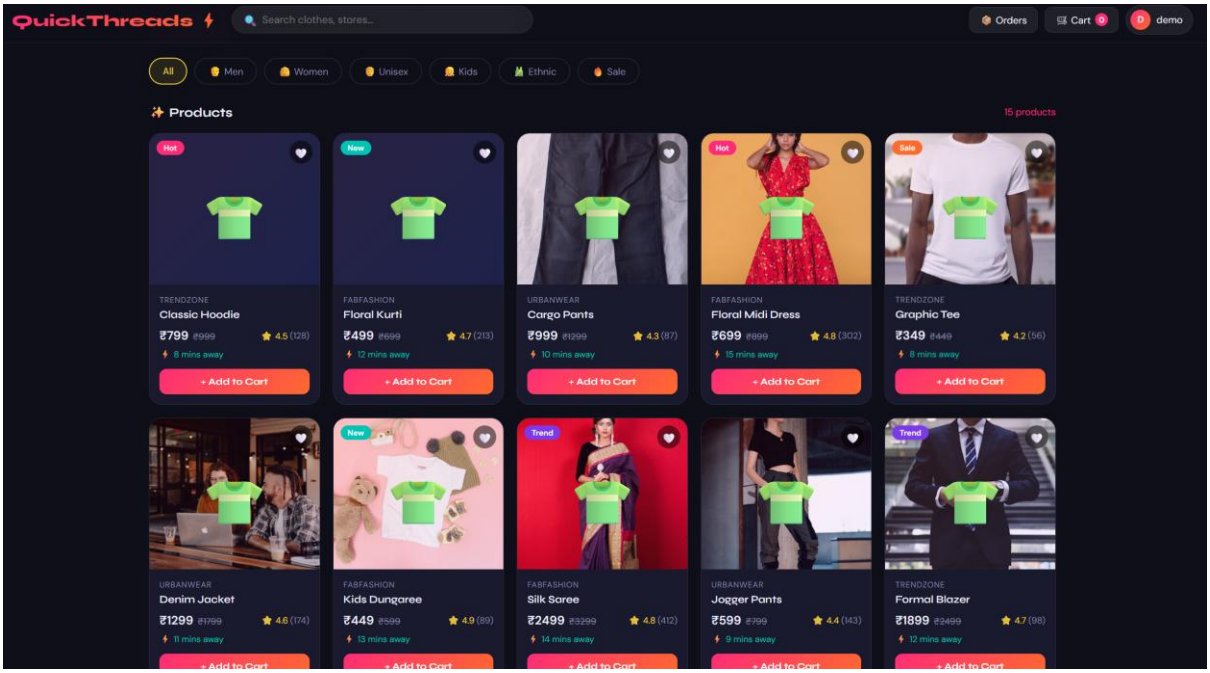
Deployed App:

1. Sign in Page

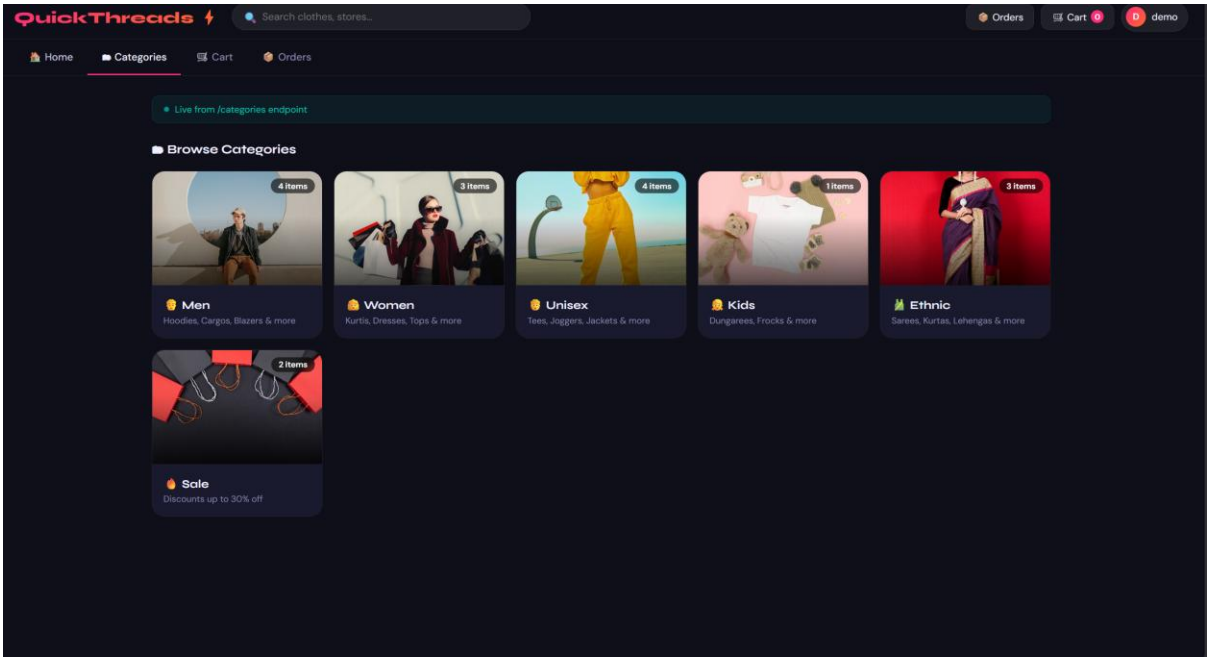


2. Home page

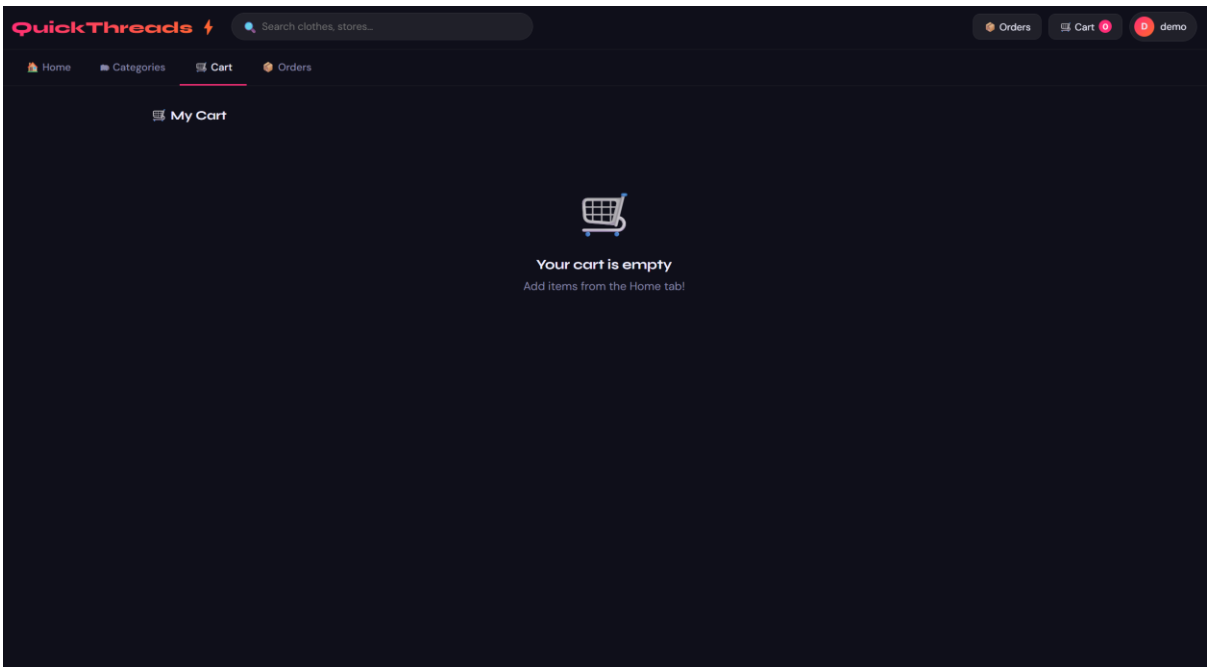

QuickThreads
Search clothes, stores...
Orders
Cart 0
D demo
Home
Categories
Cart
Orders
15 products loaded live from backend
Local Fashion,
Delivered in Minutes
Shop from TrendZone, FabFashion & UrbanWear — delivered to your door.
8–15 min delivery
3 local stores
Up to 30% OFF
15
products live
Store
All Stores
TrendZone
FabFashion
UrbanWear
All
Men
Women
Unisex
Kids
Ethnic
Sale
Products
15 products
Hot
New
Hot
Sale
Welcome to QuickThreads


QuickThreads
Search clothes, stores...
Orders
Cart 0
D demo
All
Men
Women
Unisex
Kids
Ethnic
Sale
Products
15 products
Hot
TRENDZONE
Classic Hoodie
₹799 ₹999
4.5 (128)
8 mins away
+ Add to Cart
New
FABFASHION
Floral Kurti
₹499 ₹699
4.7 (213)
12 mins away
+ Add to Cart
URBANWEAR
Cargo Pants
₹999 ₹1299
4.3 (87)
10 mins away
+ Add to Cart
Hot
FABFASHION
Floral Midi Dress
₹699 ₹899
4.8 (302)
15 mins away
+ Add to Cart
Sale
TRENDZONE
Graphic Tee
₹349 ₹449
4.2 (56)
8 mins away
+ Add to Cart
URBANWEAR
Denim Jacket
₹1299 ₹1799
4.6 (174)
11 mins away
New
FABFASHION
Kids Dungaree
₹449 ₹599
4.9 (89)
13 mins away
Trend
FABFASHION
Silk Saree
₹2499 ₹3299
4.8 (412)
14 mins away
URBANWEAR
Jogger Pants
₹599 ₹799
4.4 (143)
9 mins away
Trend
TRENDZONE
Formal Blazer
₹1899 ₹2499
4.7 (98)
12 mins away

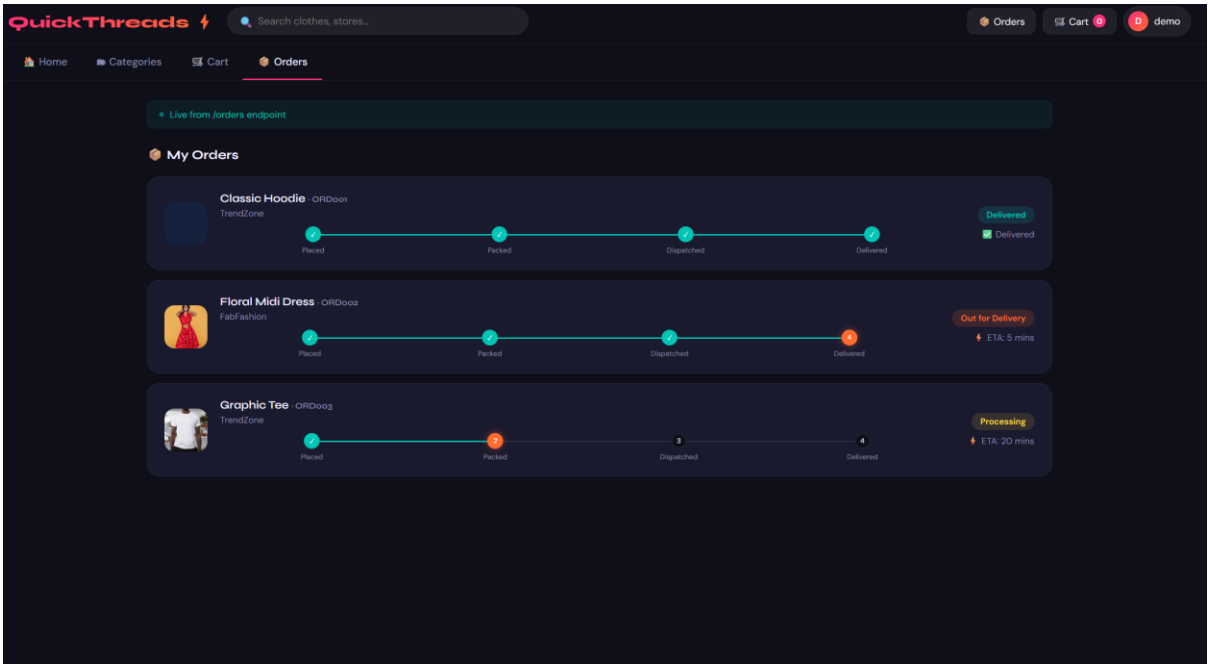
3. Categories Page



4. Cart Page



5. Orders Page



Jenkins

What is Jenkins?

Jenkins is an **automation server** used for CI/CD — **Continuous Integration and Continuous Deployment**. It automatically runs a series of steps every time new code is pushed to GitHub.

What is CI/CD?

Continuous Integration (CI) — Automatically build and test the app whenever someone pushes new code.

Continuous Deployment (CD) — Automatically deploy the latest version of the app to the server without any manual work.

How does Jenkins work?

Jenkins follows a **pipeline** defined in a file called Jenkinsfile. This file lists all the stages Jenkins should execute in order.

Our Jenkins Pipeline Stages

1. **Clone** — Jenkins pulls the latest code from GitHub automatically
2. **Build** — Jenkins builds the Docker image yourstore:latest
3. **Load** — Jenkins loads the image into Minikube
4. **Deploy** — Jenkins applies the deployment.yaml to Kubernetes
5. **Verify** — Jenkins checks that the pods are running successfully

Why are we using it?

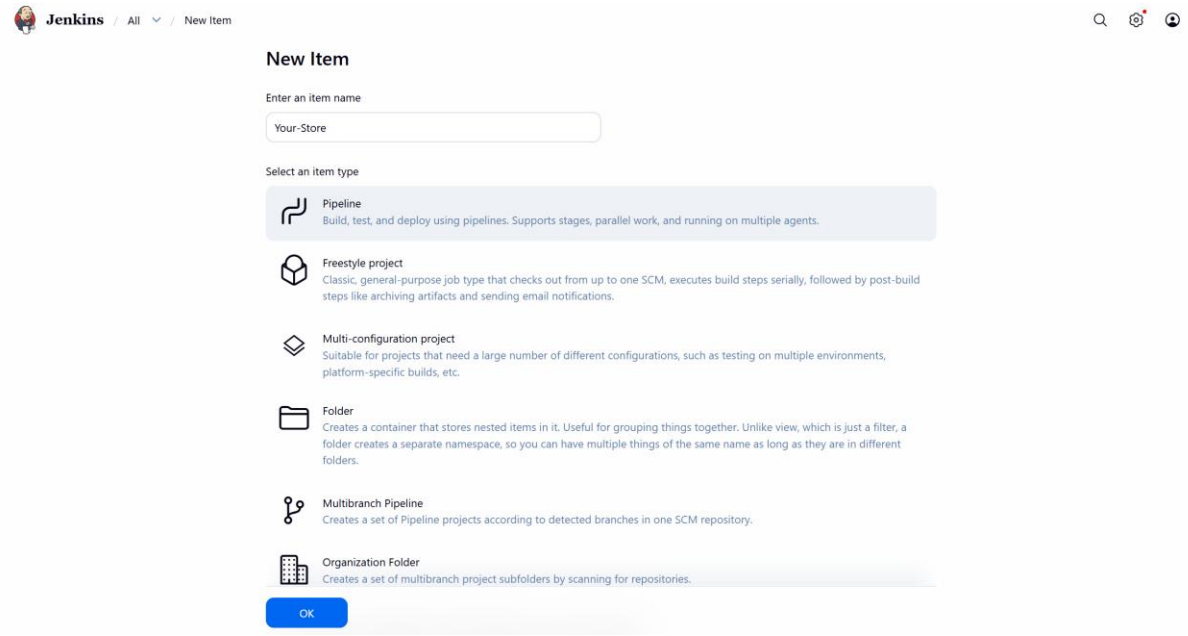
Without Jenkins, every time you update your code you have to manually build the image, load it, and redeploy. Jenkins does all of this automatically the moment you push to GitHub.

Real World Analogy

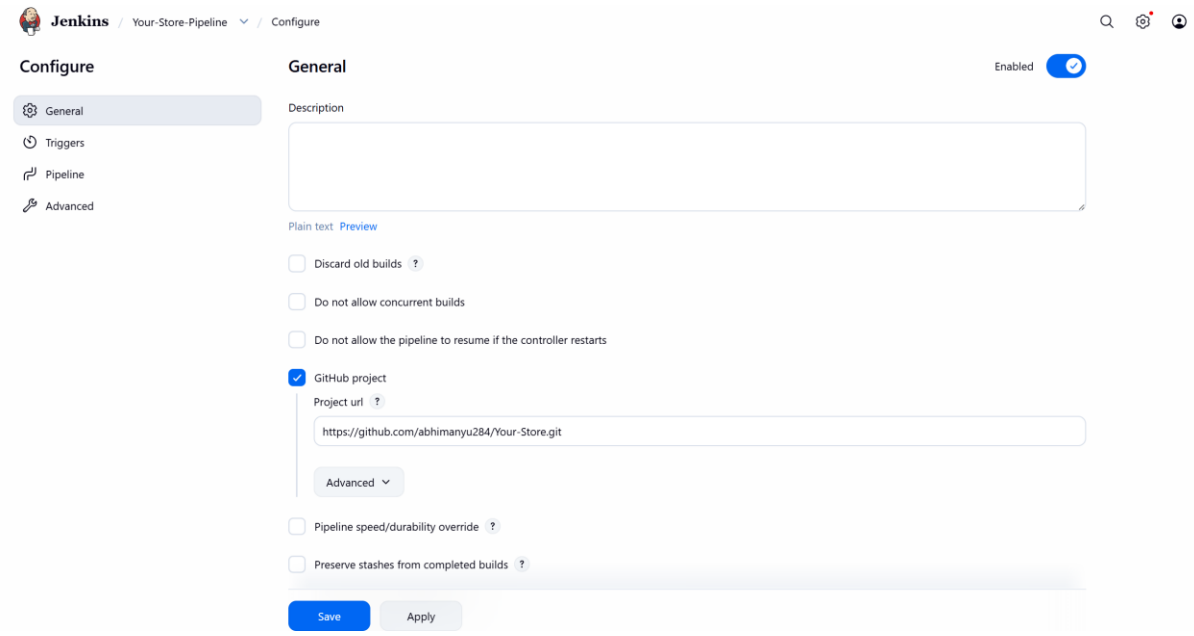
Jenkins is like a **factory assembly line**. The moment raw material (new code) arrives, the line automatically processes it — builds, tests, packages, and delivers it — without any human touching it in between.

Commands:

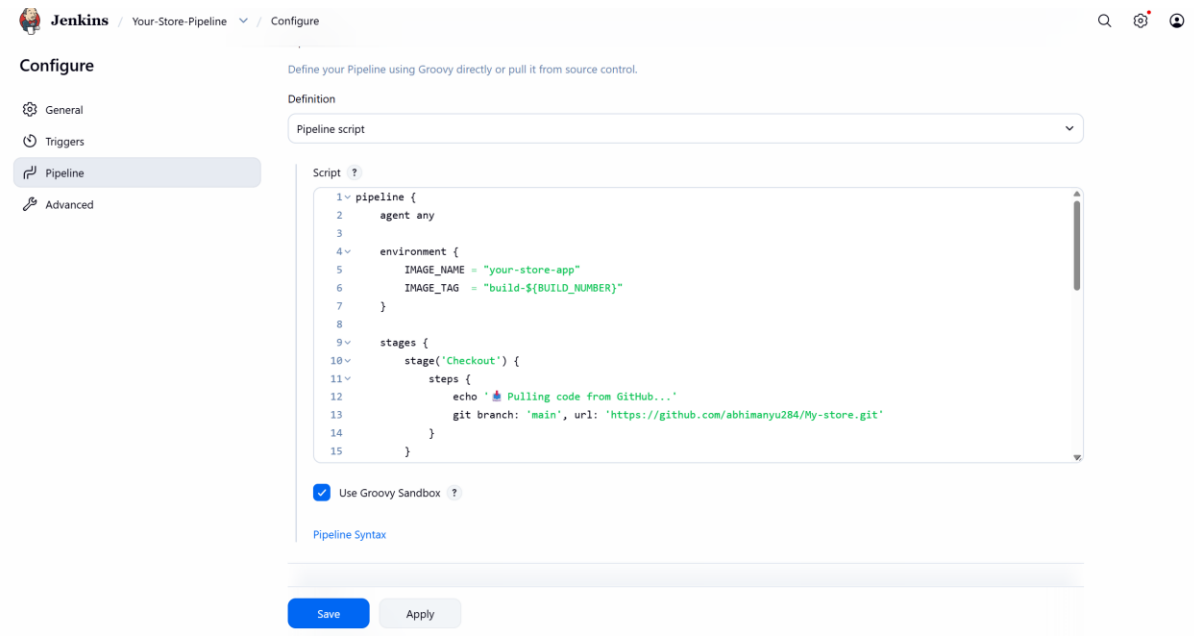
Step 1: Creating a new Pipeline



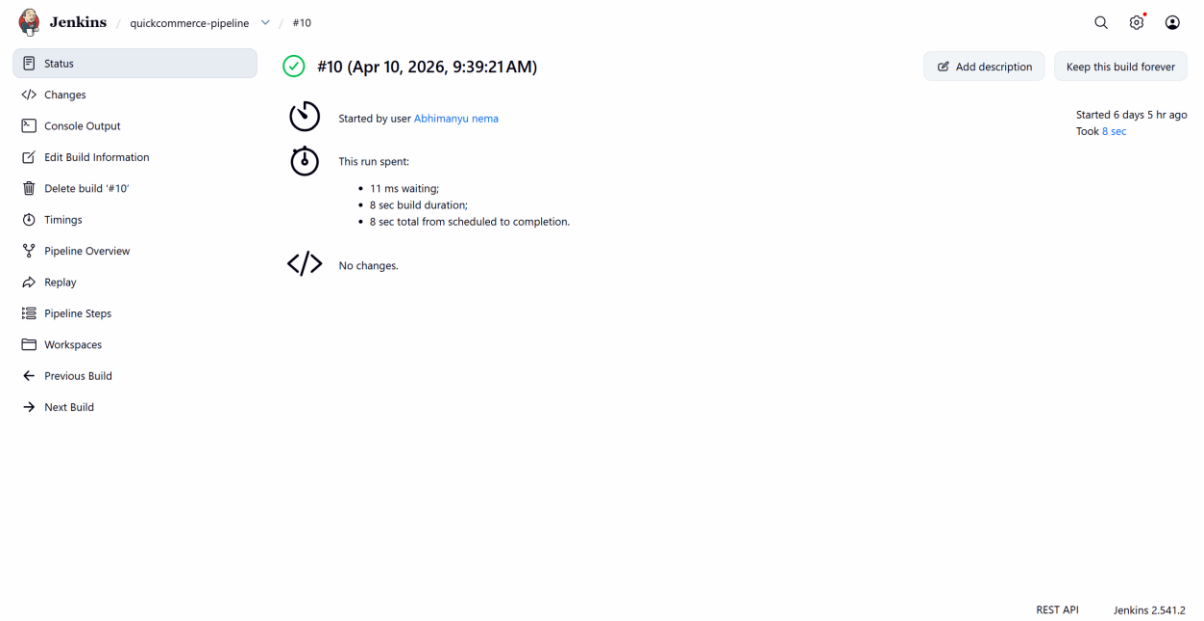
Step 2: Adding the GitHub Link



Step 3: Adding Script Code



Step 4: Saving & Building



Prometheus

What is Prometheus?
Prometheus is an open-source monitoring and alerting tool. It continuously collects data (called metrics) from your application and stores them so you can analyze the app's health and performance over time.

What are Metrics?
Metrics are numerical measurements about your app — things like how many requests it received, how fast it responded, how much memory it's using, and whether it's healthy or not.

How does Prometheus work?
Prometheus works by scraping — it visits your app's /metrics endpoint at regular intervals and collects data. Our app exposes this endpoint built into the Node.js backend.

Key Endpoints in our app

- http://localhost:3000/health — Returns whether the app is running and healthy
- http://localhost:3000/metrics — Returns all the performance data Prometheus collects

Why are we using it?
In production, you can never just *assume* your app is working. You need real data — how many users are active, is the server slow, is something crashing? Prometheus gives you that visibility.

What we did in this project

Ran Prometheus as a Docker container on port 9090
Configured it to scrape metrics from our app
Accessed the Prometheus dashboard at http://localhost:9090

Real World Analogy

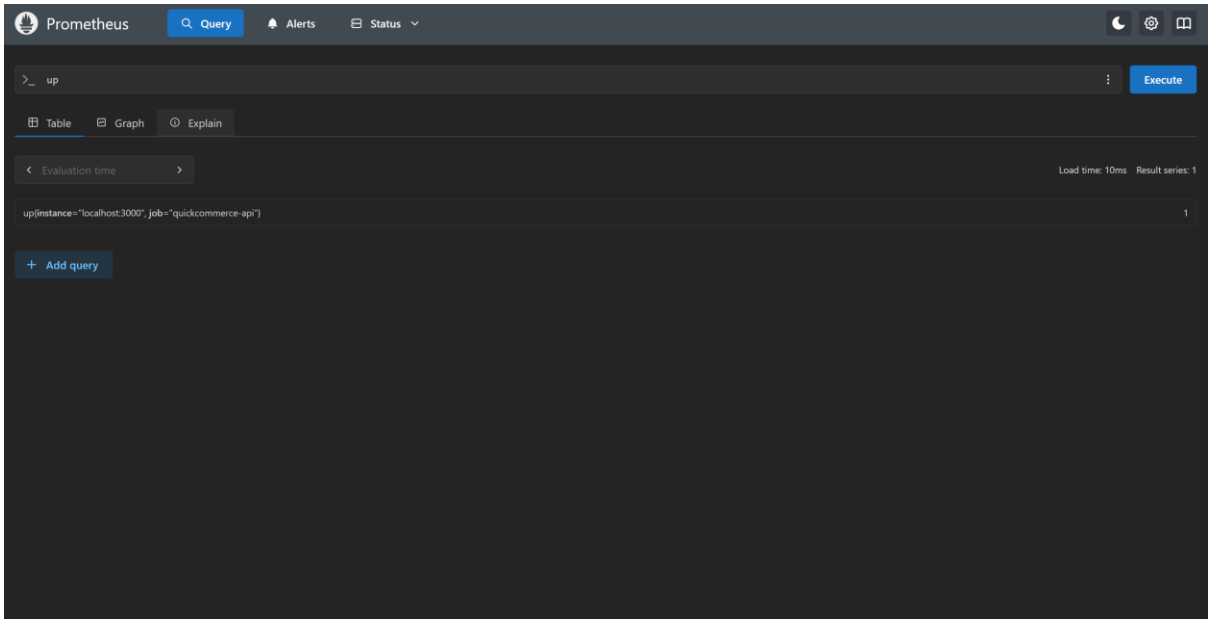
Prometheus is like the dashboard in your car — it constantly monitors speed, fuel, engine temperature, and alerts you the moment something goes wrong. You don't wait for the car to break down; you watch the dashboard.

Command:

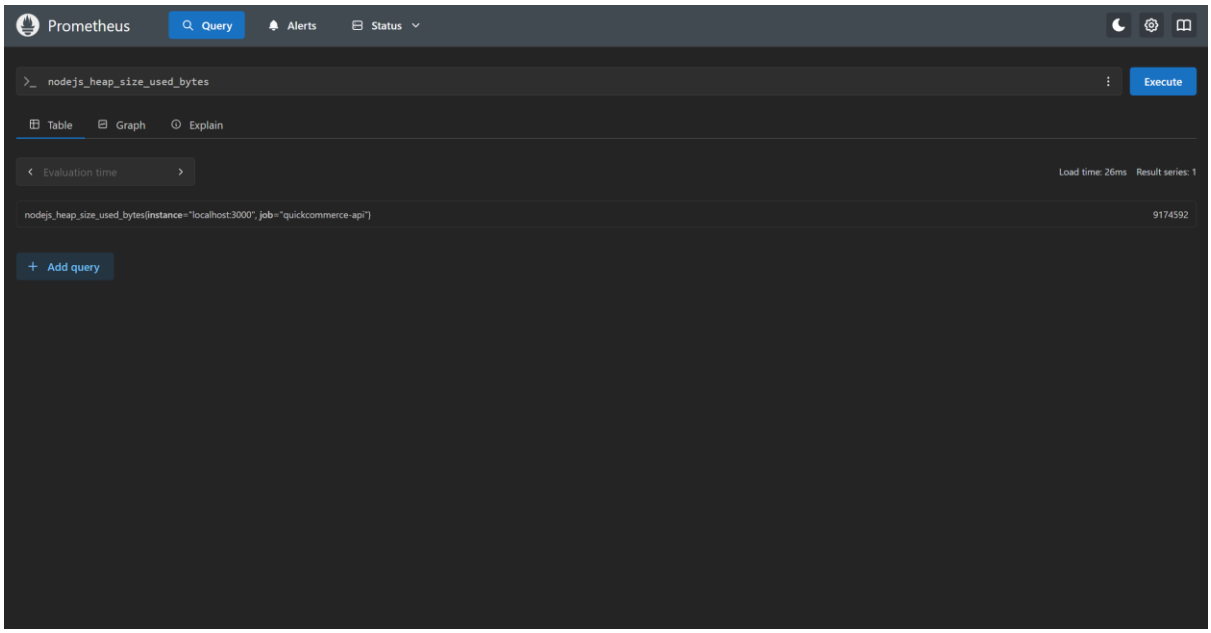
1. ./Prometheus.exe

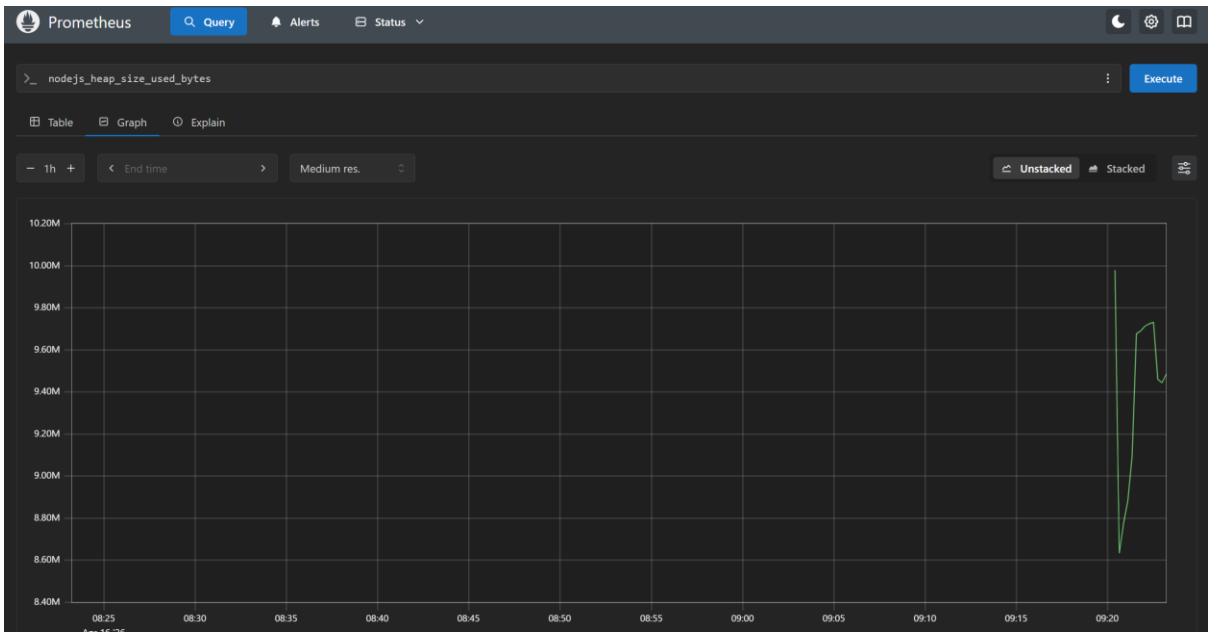
```
PS C:\Users\Abhimanyu Nema\OneDrive\Desktop\SEM 6\Azile & Product Development\prometheus-3.10.0.windows-amd64\prometheus-3.10.0.windows-amd64> ./prometheus.exe
time=2026-04-16T14:50:26.799+05:30 level=INFO source=main.go:1611 msg="updated GOGC" old=100 new=75
time=2026-04-16T14:50:26.801+05:30 level=INFO source=main.go:723 msg="Leaving GOMAXPROCS=20: CPU quota undefined" component=automaxprocs
time=2026-04-16T14:50:26.802+05:30 level=INFO source=memlimit.go:198 msg="GOMEMLIMIT is updated" component=automemlimit package=github.com/KimMachineGun/automemlimit/memlimit GOME
MLIMIT=15201290649 previous=9223372036854775807
time=2026-04-16T14:50:26.802+05:30 level=INFO source=main.go:771 msg="No time or size retention was set so using the default time retention" duration=15d
time=2026-04-16T14:50:26.804+05:30 level=INFO source=main.go:822 msg="Starting Prometheus Server" mode=server version="(version=3.10.0, branch=HEAD, revision=54e010926b0a270cadb22
be1113ad45fe9bcb90a)"
time=2026-04-16T14:50:26.804+05:30 level=INFO source=main.go:827 msg="operational information" build_context="(go=go1.26.0, platform=windows/amd64, user=root@a38ea3653297, date=20
260225-21:17:09, tags=builtinassets)" host_details=(windows) fd_limits=N/A vm_limits=N/A
time=2026-04-16T14:50:27.041+05:30 level=INFO source=web.go:696 msg="Start listening for connections" component=web address=0.0.0.0:9090
time=2026-04-16T14:50:27.045+05:30 level=INFO source=main.go:1350 msg="Starting TSDB ..."
time=2026-04-16T14:50:27.047+05:30 level=INFO source=repair.go:54 msg="Found healthy block" component=tsdb mint=1775756746960 maxt=1775757600000 ulid=01KNTFDY1T8D8BVH8TH4AY4K05
time=2026-04-16T14:50:27.048+05:30 level=INFO source=repair.go:54 msg="Found healthy block" component=tsdb mint=1775884866670 maxt=1775887200000 ulid=01KPASCTA5YWC6X65Q0HKJWV7E
time=2026-04-16T14:50:27.048+05:30 level=INFO source=tls_config.go:354 msg="Listening on" component=web address=[::]:9090
time=2026-04-16T14:50:27.048+05:30 level=INFO source=tls_config.go:357 msg="TLS is disabled." component=web http2=false address=[::]:9090
time=2026-04-16T14:50:27.048+05:30 level=INFO source=repair.go:54 msg="Found healthy block" component=tsdb mint=1775887201670 maxt=1775894400000 ulid=01KPASCTDQ8QS7KBH049DW8PXV
time=2026-04-16T14:50:27.050+05:30 level=INFO source=repair.go:54 msg="Found healthy block" component=tsdb mint=1775783835651 maxt=1775808000000 ulid=01KPASCTP038E2ER35S26X6G0T
time=2026-04-16T14:50:27.062+05:30 level=INFO source=head.go:680 msg="Replaying on-disk memory mappable chunks if any" component=tsdb
time=2026-04-16T14:50:27.063+05:30 level=INFO source=head.go:766 msg="On-disk memory mappable chunks replay completed" component=tsdb duration=0s
time=2026-04-16T14:50:27.063+05:30 level=INFO source=head.go:774 msg="Replaying WAL, this may take a while" component=tsdb
time=2026-04-16T14:50:27.065+05:30 level=INFO source=head.go:811 msg="WAL checkpoint loaded" component=tsdb
time=2026-04-16T14:50:27.071+05:30 level=INFO source=head.go:847 msg="WAL segment loaded" component=tsdb segment=23 maxSegment=26 duration=6.1989ms
time=2026-04-16T14:50:27.074+05:30 level=INFO source=head.go:847 msg="WAL segment loaded" component=tsdb segment=24 maxSegment=26 duration=2.9987ms
time=2026-04-16T14:50:27.078+05:30 level=INFO source=head.go:847 msg="WAL segment loaded" component=tsdb segment=25 maxSegment=26 duration=3.5208ms
time=2026-04-16T14:50:27.078+05:30 level=INFO source=head.go:847 msg="WAL segment loaded" component=tsdb segment=26 maxSegment=26 duration=0s
time=2026-04-16T14:50:27.078+05:30 level=INFO source=head.go:884 msg="WAL replay completed" component=tsdb checkpoint_replay_duration=2.3364ms wal_replay_duration=12.7184ms wbl_re
play_duration=0s chunk_snapshot_load_duration=0s mmap_chunk_replay_duration=0s total_replay_duration=15.0548ms
time=2026-04-16T14:50:27.081+05:30 level=INFO source=main.go:1371 msg="filesystem information" fs_type=unknown
time=2026-04-16T14:50:27.081+05:30 level=INFO source=main.go:1374 msg="TSDB started"
time=2026-04-16T14:50:27.082+05:30 level=INFO source=main.go:1564 msg="Loading configuration file" filename=prometheus.yml
time=2026-04-16T14:50:27.116+05:30 level=INFO source=main.go:1604 msg="Completed loading of configuration file" db_storage=0s remote_storage=0s web_handler=0s query_engine=0s scra
pe=33.5698ms scrape_sd=0s notify=0s notify_sd=0s rules=0s tracing=0s filename=prometheus.yml totalDuration=33.9952ms
time=2026-04-16T14:50:27.117+05:30 level=INFO source=main.go:1335 msg="Server is ready to receive web requests."
time=2026-04-16T14:50:27.117+05:30 level=INFO source=manager.go:202 msg="Starting rule manager..." component="rule manager"
time=2026-04-16T14:50:35.683+05:30 level=INFO source=compact.go:590 msg="write block started" component=tsdb mint=1775894400000 maxt=1775901600000 ulid=01KPASF8B34SQD60EWM4GQ0DMF
time=2026-04-16T14:50:35.739+05:30 level=INFO source=compact.go:618 msg="write block resulted in empty block" component=tsdb mint=1775894400000 maxt=1775901600000 duration=55.8501
ms
time=2026-04-16T14:50:35.741+05:30 level=INFO source=head.go:1453 msg="Head GC started" component=tsdb caller=truncateMemory
time=2026-04-16T14:50:35.744+05:30 level=INFO source=head.go:1457 msg="Head GC completed" component=tsdb caller=truncateMemory duration=2.7455ms
time=2026-04-16T14:50:35.745+05:30 level=INFO source=checkpoint.go:99 msg="Creating checkpoint" component=tsdb from_segment=23 to_segment=24 mint=1775901600000
time=2026-04-16T14:50:35.768+05:30 level=INFO source=head.go:1418 msg="WAL checkpoint complete" component=tsdb first=23 last=24 duration=23.7947ms
```

2. Up



3. nodejs_heap_size_used_bytes





Conclusion

The Your Store project successfully demonstrates the implementation of a complete, production-grade DevOps pipeline for a real-world quick commerce application targeting local clothing stores. By integrating industry-standard tools — Git/GitHub, Docker, Kubernetes, Jenkins, and Prometheus — the project showcases how modern software development practices can be applied even at a small business scale.

Through this project, we achieved seamless version control and collaboration using GitHub, consistent and portable deployment using Docker containerization, automated build and deployment cycles through Jenkins CI/CD, resilient and scalable container orchestration via Kubernetes with live pod scaling, and real-time application monitoring using Prometheus metrics.

The pipeline eliminates manual intervention at every stage — from code push to deployment to monitoring — mirroring how top technology companies manage their production systems. This project not only demonstrates technical proficiency but also highlights the importance of automation, reliability, and observability in modern software delivery.

As a next step, integrating Grafana for visual dashboards, deploying to a cloud provider like AWS or GCP, and implementing Horizontal Pod Autoscaling (HPA) in Kubernetes would take this pipeline to an enterprise-level setup.